

PRÁCTICA COMPLETA SQL — “Tienda FP TechStore”

Objetivo

Crear una base de datos para una tienda y resolver consultas reales de negocio.

1) Script completo de creación e inserción (ejecútalo tal cual)

```
/* =====  
PRACTICA SQL COMPLETA - TechStore (FP)  
MySQL / MariaDB - Importable en HeidiSQL  
===== */
```

```
DROP DATABASE IF EXISTS techstore_fp;
```

```
CREATE DATABASE techstore_fp CHARACTER SET utf8mb4 COLLATE  
utf8mb4_unicode_ci;
```

```
USE techstore_fp;
```

```
SET FOREIGN_KEY_CHECKS=0;
```

```
-- ----- TABLAS -----
```

```
DROP TABLE IF EXISTS detalle_venta;
```

```
DROP TABLE IF EXISTS ventas;
```

```
DROP TABLE IF EXISTS productos;
```

```
DROP TABLE IF EXISTS categorias;
```

```
DROP TABLE IF EXISTS clientes;
```

```
CREATE TABLE clientes (
```

```
    id_cliente INT AUTO_INCREMENT PRIMARY KEY,
```

```
    nombre VARCHAR(60) NOT NULL,
```

```
    email VARCHAR(80),
```

```
telefono VARCHAR(15),  
ciudad VARCHAR(40),  
fecha_alta DATE NOT NULL  
);
```

```
CREATE TABLE categorias (  
id_categoria INT AUTO_INCREMENT PRIMARY KEY,  
nombre VARCHAR(40) NOT NULL UNIQUE  
);
```

```
CREATE TABLE productos (  
id_producto INT AUTO_INCREMENT PRIMARY KEY,  
nombre VARCHAR(80) NOT NULL,  
precio DECIMAL(10,2) NOT NULL,  
stock INT NOT NULL DEFAULT 0,  
id_categoria INT NOT NULL,  
FOREIGN KEY (id_categoria) REFERENCES categorias(id_categoria)  
);
```

```
CREATE TABLE ventas (  
id_venta INT AUTO_INCREMENT PRIMARY KEY,  
fecha DATE NOT NULL,  
id_cliente INT NOT NULL,  
metodo_pago ENUM('EFECTIVO','TARJETA','BIZUM') NOT NULL,  
FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)  
);
```

```
CREATE TABLE detalle_venta (  
id_venta INT NOT NULL,
```

```
id_producto INT NOT NULL,  
cantidad INT NOT NULL CHECK (cantidad > 0),  
precio_unitario DECIMAL(10,2) NOT NULL,  
PRIMARY KEY (id_venta, id_producto),  
FOREIGN KEY (id_venta) REFERENCES ventas(id_venta),  
FOREIGN KEY (id_producto) REFERENCES productos(id_producto)  
);
```

```
SET FOREIGN_KEY_CHECKS=1;
```

```
-- ----- DATOS -----
```

```
INSERT INTO clientes (nombre,email,telefono,ciudad,fecha_alta) VALUES  
( 'Ana Ruiz','ana@correo.com','600111222','Málaga','2025-01-10'),  
( 'Luis Pérez','luis@correo.com','600222333','Sevilla','2025-02-05'),  
( 'María Díaz','maria@correo.com','600333444','Málaga','2025-03-12'),  
( 'Javier Soto',NULL,'600444555','Granada','2025-03-20'),  
( 'Carmen León','carmen@correo.com','600555666','Cádiz','2025-04-02');
```

```
INSERT INTO categorias (nombre) VALUES  
( 'Smartphones'),  
( 'Portátiles'),  
( 'Accesorios'),  
( 'Componentes');
```

```
INSERT INTO productos (nombre,precio,stock,id_categoria) VALUES  
( 'iPhone 13',599.00,10,1),  
( 'Samsung A54',329.00,15,1),  
( 'Xiaomi Redmi Note 12',199.00,20,1),  
( 'Portátil Lenovo i5',749.00,8,2),
```

```
('MacBook Air M1',899.00,5,2),  
( 'Auriculares Bluetooth',49.90,50,3),  
( 'Cargador USB-C 65W',29.90,40,3),  
( 'SSD 1TB',89.90,30,4),  
( 'RAM 16GB DDR4',59.90,25,4);
```

```
INSERT INTO ventas (fecha,id_cliente,metodo_pago) VALUES  
( '2025-04-10',1,'TARJETA'),  
( '2025-04-10',2,'BIZUM'),  
( '2025-04-12',1,'EFECTIVO'),  
( '2025-04-15',3,'TARJETA'),  
( '2025-04-18',5,'TARJETA');
```

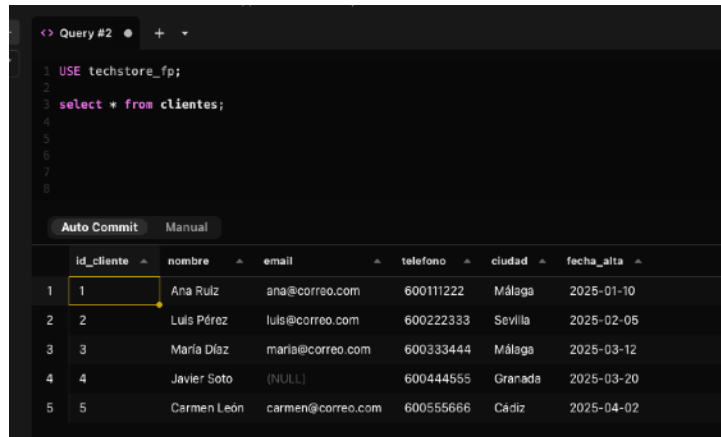
-- detalle: (id_venta, id_producto, cantidad, precio_unitario)

```
INSERT INTO detalle_venta VALUES  
(1,1,1,599.00),  
(1,6,2,49.90),  
(2,3,1,199.00),  
(2,7,1,29.90),  
(3,2,1,329.00),  
(3,6,1,49.90),  
(4,4,1,749.00),  
(4,8,1,89.90),  
(5,5,1,899.00),  
(5,7,2,29.90);
```

2) Batería de consultas (resuelve y entrega)

Nivel 1 — Consultas sencillas

1. Lista todos los clientes.

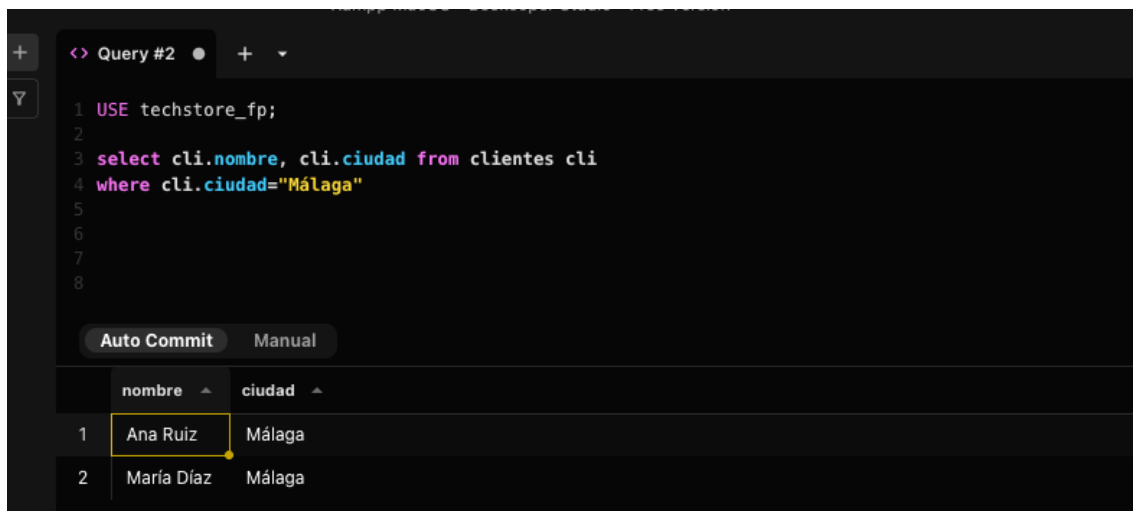


```
Query #2
1 USE techstore_fp;
2
3 select * from clientes;
4
5
6
7
8
```

Auto Commit Manual

	id_cliente	nombre	email	telefono	ciudad	fecha_alta
1	1	Ana Ruiz	ana@correo.com	600111222	Málaga	2025-01-10
2	2	Luis Pérez	luis@correo.com	600222333	Sevilla	2025-02-05
3	3	María Díaz	maria@correo.com	600333444	Málaga	2025-03-12
4	4	Javier Soto	(NULL)	600444555	Granada	2025-03-20
5	5	Carmen León	carmen@correo.com	600555666	Cádiz	2025-04-02

2. Muestra nombre y ciudad de los clientes de Málaga.

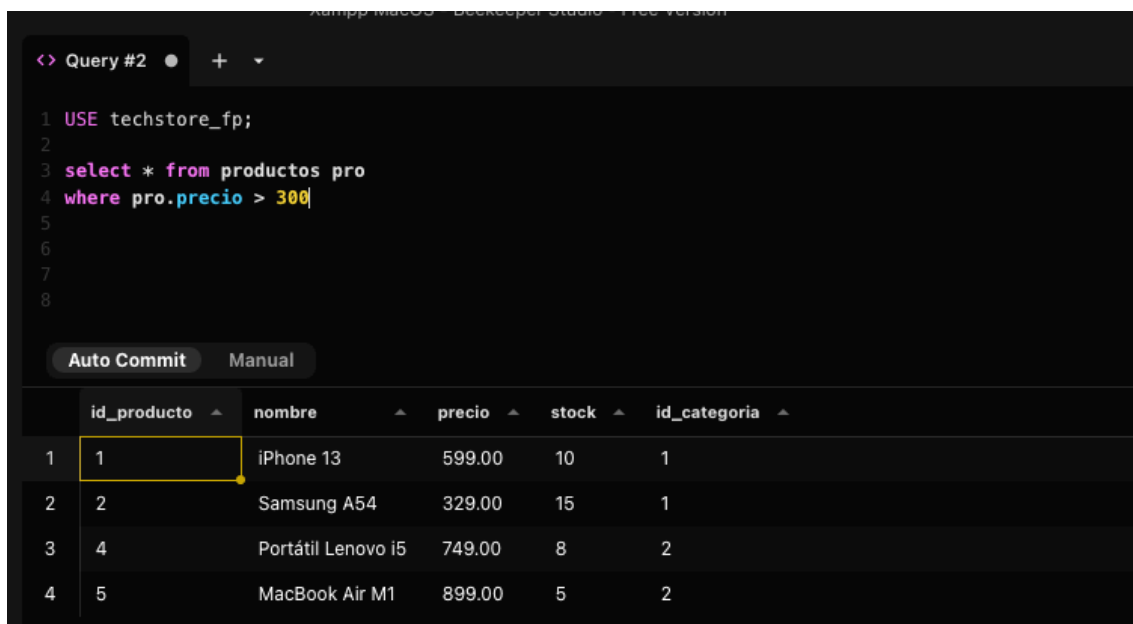


```
Query #2
1 USE techstore_fp;
2
3 select cli.nombre, cli.ciudad from clientes cli
4 where cli.ciudad="Málaga"
5
6
7
8
```

Auto Commit Manual

	nombre	ciudad
1	Ana Ruiz	Málaga
2	María Díaz	Málaga

3. Lista productos con precio > 300 ordenados de mayor a menor.



```
Query #2
1 USE techstore_fp;
2
3 select * from productos pro
4 where pro.precio > 300
5
6
7
8
```

Auto Commit Manual

	id_producto	nombre	precio	stock	id_categoria
1	1	iPhone 13	599.00	10	1
2	2	Samsung A54	329.00	15	1
3	4	Portátil Lenovo i5	749.00	8	2
4	5	MacBook Air M1	899.00	5	2

4. Muestra los 3 productos más baratos.

Xampp MacOS - Beekeeper Studio - Free Version

```
<> Query #2 ● + -
1 USE techstore_fp;
2
3 select * from productos pro
4 order by pro.precio asc
5 limit 3;
6
7
8
```

Auto Commit Manual

	id_producto ▲	nombre ▲	precio ▲	stock ▲	id_categoria ▲
1	7	Cargador USB-...	29.90	40	3
2	6	Auriculares Blu...	49.90	50	3
3	9	RAM 16GB DDR4	59.90	25	4

5. Cuenta cuántos productos hay por categoría.

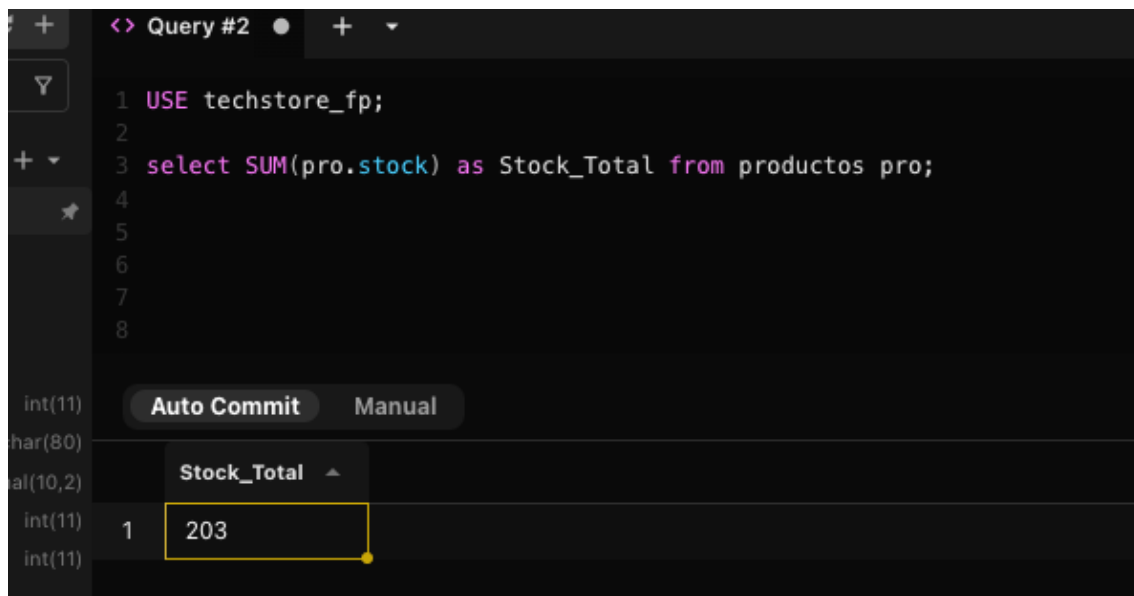
Xampp MacOS - Beekeeper Studio - Free Version

```
+
-
<> Query #2 ● + -
1 USE techstore_fp;
2
3 select count(c.id_categoria) as N°_Productos, c.nombre from productos pro
4 join categorias c on c.id_categoria=pro.id_categoria
5 group by c.nombre
6
7
8
```

Auto Commit Manual

	N°_Productos ▲	nombre ▲
1	2	Accesorios
2	2	Componentes
3	2	Portátiles
4	3	Smartphones

6. Muestra el stock total (suma de stock) de todos los productos.



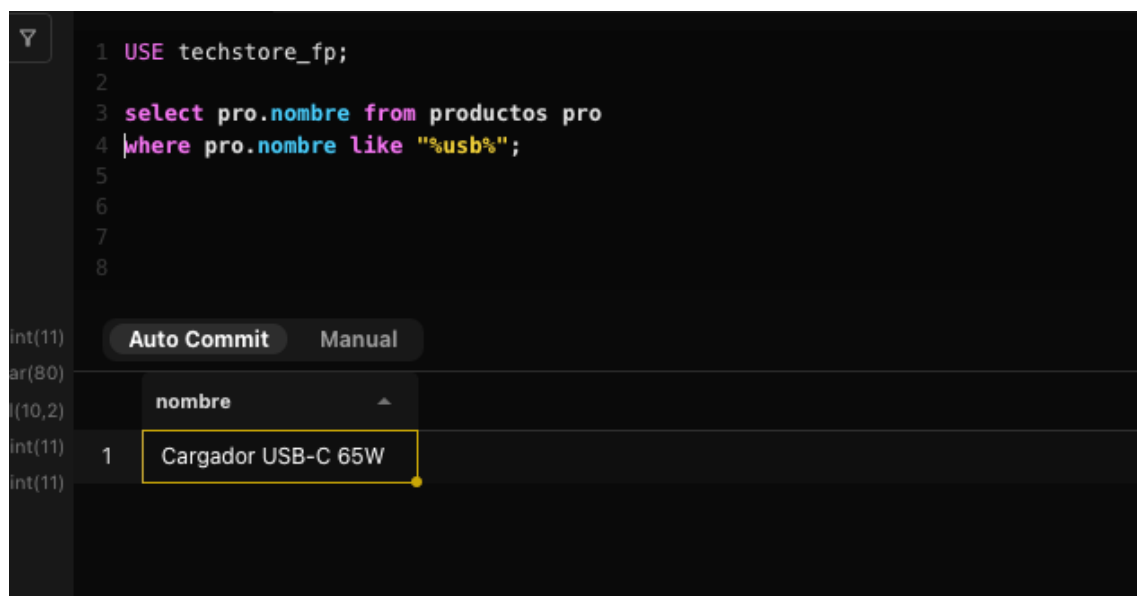
The screenshot shows a SQL query editor with the following code:

```
1 USE techstore_fp;
2
3 select SUM(pro.stock) as Stock_Total from productos pro;
```

Below the query editor, there are two buttons: "Auto Commit" and "Manual". The results table shows a single row with the value 203.

	Stock_Total
1	203

7. Muestra productos cuyo nombre contenga "USB".



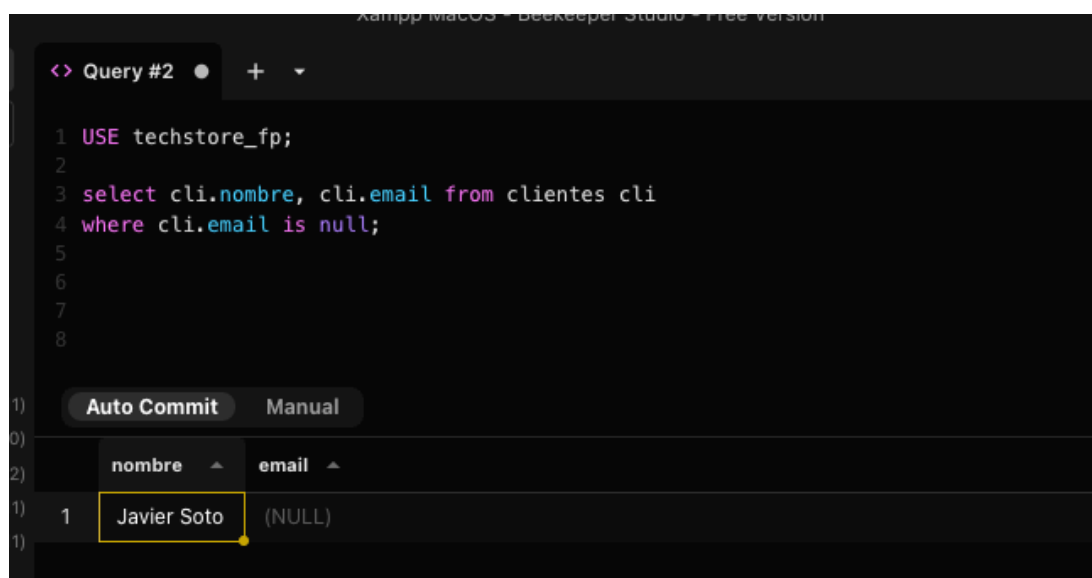
The screenshot shows a SQL query editor with the following code:

```
1 USE techstore_fp;
2
3 select pro.nombre from productos pro
4 where pro.nombre like "%usb%";
```

Below the query editor, there are two buttons: "Auto Commit" and "Manual". The results table shows a single row with the value "Cargador USB-C 65W".

	nombre
1	Cargador USB-C 65W

8. Muestra clientes con email NULL.



The screenshot shows a SQL query editor with the following code:

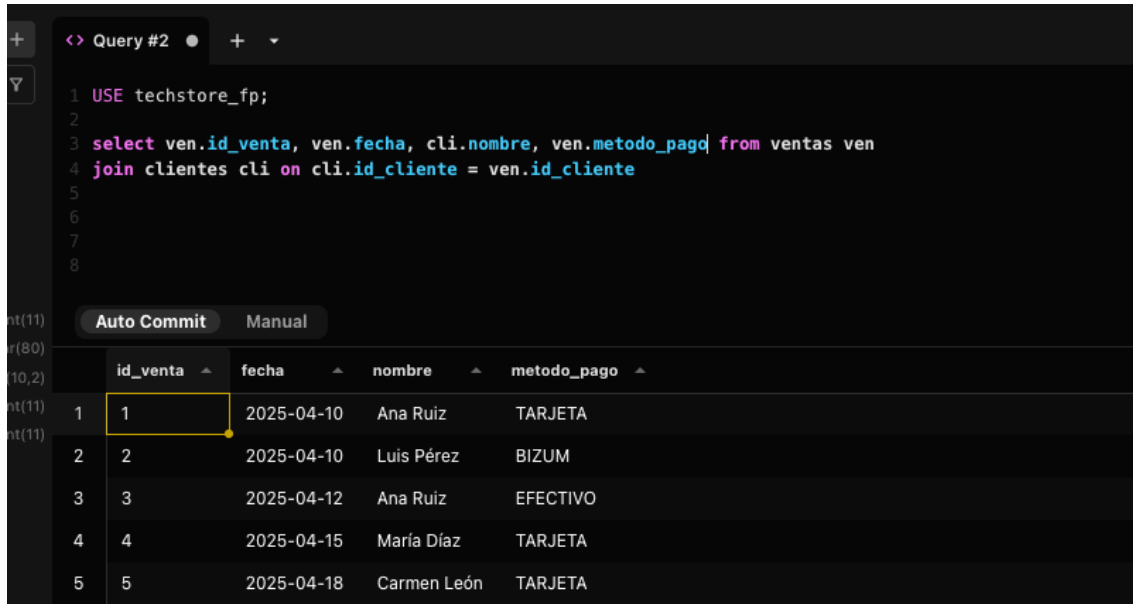
```
1 USE techstore_fp;
2
3 select cli.nombre, cli.email from clientes cli
4 where cli.email is null;
```

Below the query editor, there are two buttons: "Auto Commit" and "Manual". The results table shows a single row with the value "Javier Soto" and "(NULL)".

	nombre	email
1	Javier Soto	(NULL)

🟡 Nivel 2 — INNER JOIN (consultas reales)

9. Lista ventas con: id_venta, fecha, cliente, método de pago.



The screenshot shows a SQL query editor with a query titled "Query #2". The query is as follows:

```
1 USE techstore_fp;
2
3 select ven.id_venta, ven.fecha, cli.nombre, ven.metodo_pago from ventas ven
4 join clientes cli on cli.id_cliente = ven.id_cliente
5
6
7
8
```

Below the query, there are two buttons: "Auto Commit" and "Manual". The results of the query are displayed in a table with the following columns: id_venta, fecha, nombre, and metodo_pago. The results are as follows:

	id_venta	fecha	nombre	metodo_pago
1	1	2025-04-10	Ana Ruiz	TARJETA
2	2	2025-04-10	Luis Pérez	BIZUM
3	3	2025-04-12	Ana Ruiz	EFFECTIVO
4	4	2025-04-15	María Díaz	TARJETA
5	5	2025-04-18	Carmen León	TARJETA

10. Saca el detalle de una venta: id_venta, producto, cantidad, precio_unitario.



The screenshot shows a SQL query editor with a query titled "Query #2". The query is as follows:

```
1 USE techstore_fp;
2
3 select dv.id_venta, pro.nombre, dv.cantidad, dv.precio_unitario from detalle_venta dv
4 join productos pro on pro.id_producto=dv.id_producto
5 limit 1
6
7
8
```

Below the query, there are two buttons: "Auto Commit" and "Manual". The results of the query are displayed in a table with the following columns: id_venta, nombre, cantidad, and precio_unitario. The results are as follows:

	id_venta	nombre	cantidad	precio_unitario
1	1	iPhone 13	1	599.00

11. Calcula el importe por línea (cantidad * precio_unitario).

Xampp MacOS - Beekeeper Studio - Free Version

```
1 USE techstore_fp;
2
3 select dv.id_venta, pro.nombre, dv.cantidad, dv.precio_unitario,
4       (dv.cantidad * dv.precio_unitario) as Total
5   from detalle_venta dv
6  join productos pro on pro.id_producto=dv.id_producto
7
8
```

Auto Commit Manual

	id_venta	nombre	cantidad	precio_unitario	Total
1	1	iPhone 13	1	599.00	599.00
2	1	Auriculares Bluetooth	2	49.90	99.80
3	2	Xiaomi Redmi Note 12	1	199.00	199.00
4	2	Cargador USB-C 65W	1	29.90	29.90

12. Calcula el total de cada venta.

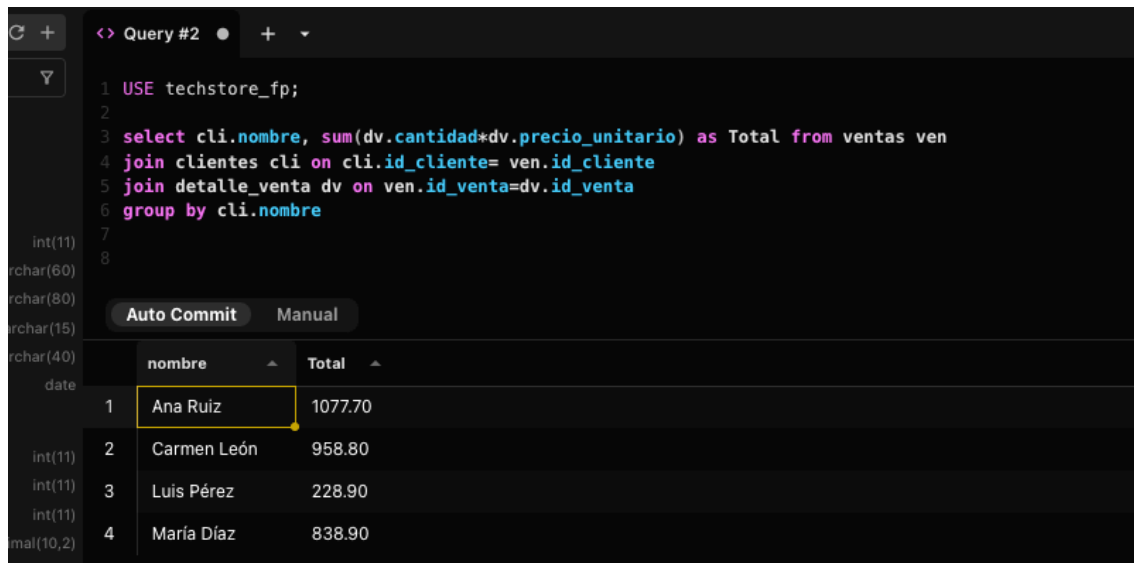
Xampp MacOS - Beekeeper Studio - Free Version

```
1 USE techstore_fp;
2
3 select dv.id_venta,
4       sum(dv.cantidad * dv.precio_unitario) as Total
5   from detalle_venta dv
6  group by dv.id_venta;
7
8
```

Auto Commit Manual

	id_venta	total_venta
1	1	698.80
2	2	228.90
3	3	378.90
4	4	838.90
5	5	958.80

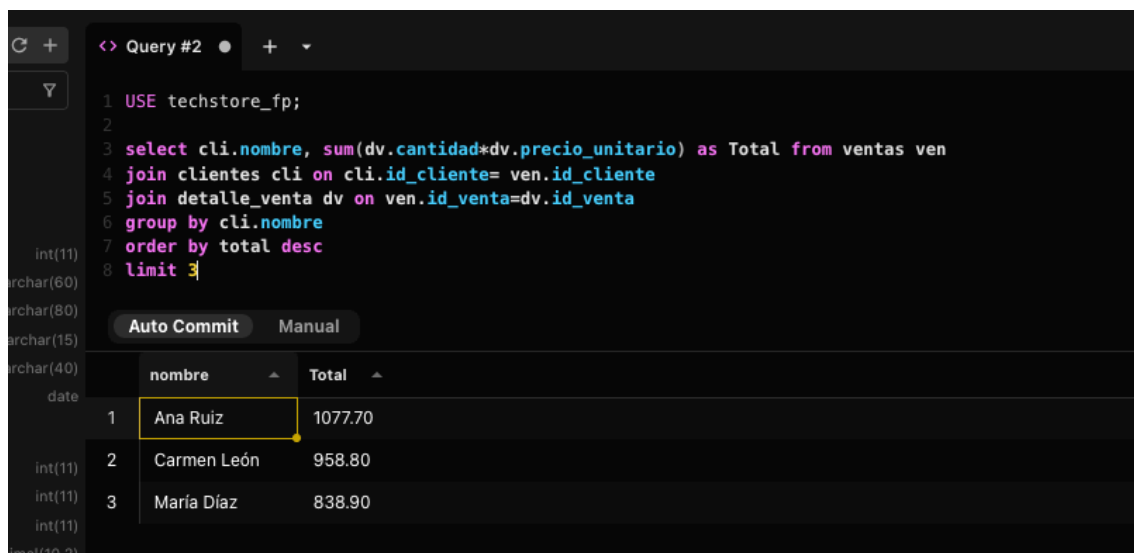
13. Calcula el total gastado por cada cliente.



```
1 USE techstore_fp;
2
3 select cli.nombre, sum(dv.cantidad*dv.precio_unitario) as Total from ventas ven
4 join clientes cli on cli.id_cliente= ven.id_cliente
5 join detalle_venta dv on ven.id_venta=dv.id_venta
6 group by cli.nombre
```

	nombre	Total
1	Ana Ruiz	1077.70
2	Carmen León	958.80
3	Luis Pérez	228.90
4	María Díaz	838.90

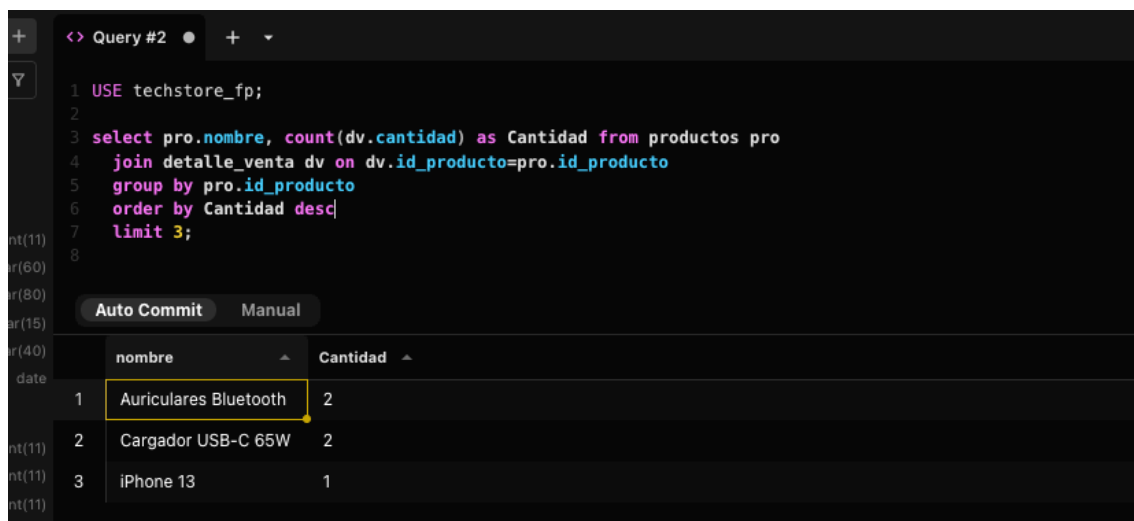
14. Saca el TOP 3 clientes que más han gastado.



```
1 USE techstore_fp;
2
3 select cli.nombre, sum(dv.cantidad*dv.precio_unitario) as Total from ventas ven
4 join clientes cli on cli.id_cliente= ven.id_cliente
5 join detalle_venta dv on ven.id_venta=dv.id_venta
6 group by cli.nombre
7 order by total desc
8 limit 3
```

	nombre	Total
1	Ana Ruiz	1077.70
2	Carmen León	958.80
3	María Díaz	838.90

15. Saca el TOP 3 productos más vendidos (por unidades).



```
1 USE techstore_fp;
2
3 select pro.nombre, count(dv.cantidad) as Cantidad from productos pro
4 join detalle_venta dv on dv.id_producto=pro.id_producto
5 group by pro.id_producto
6 order by Cantidad desc
7 limit 3;
```

	nombre	Cantidad
1	Auriculares Bluetooth	2
2	Cargador USB-C 65W	2
3	iPhone 13	1

16. Saca facturación por día.

xampp MacOS - Beekeeper Studio - Free version

Query #2

```
1 USE techstore_fp;
2
3 select ven.fecha, sum(dv.cantidad*dv.precio_unitario) As Facturacion_Día from ventas ven
4   join detalle_venta dv on dv.id_venta=ven.id_venta
5  group by ven.fecha
6
7
8
```

Auto Commit Manual

	fecha	Facturacion_Día
1	2025-04-10	927.70
2	2025-04-12	378.90
3	2025-04-15	838.90
4	2025-04-18	958.80

16.1 delete

Delete from where

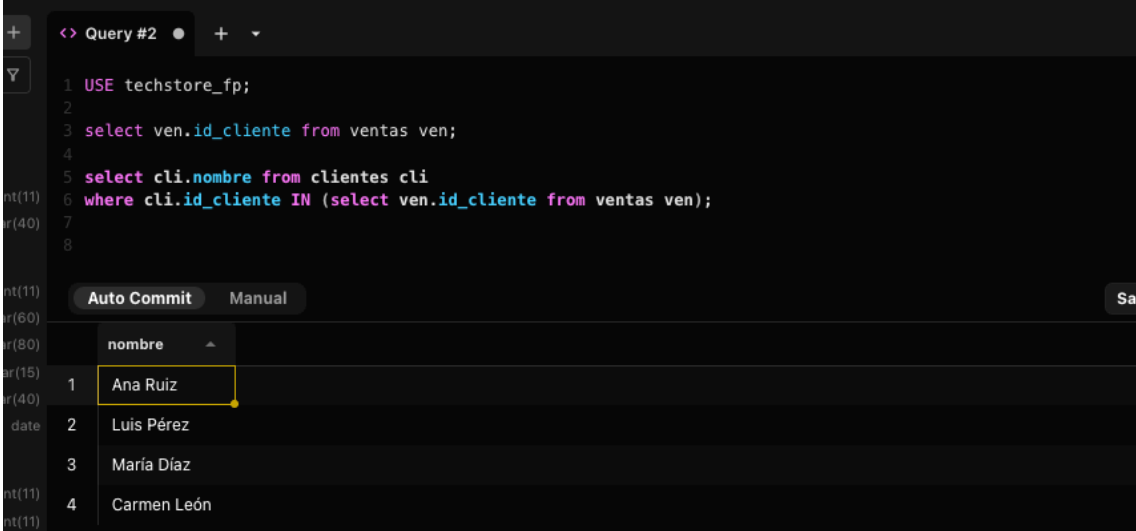
16.2 update

Update nombre_table set columna where

Update alumnos set edad=21 where id=12

Nivel 3 — Subconsultas (IN / NOT IN / AVG / EXISTS)

17. Clientes que han comprado al menos una vez (IN).

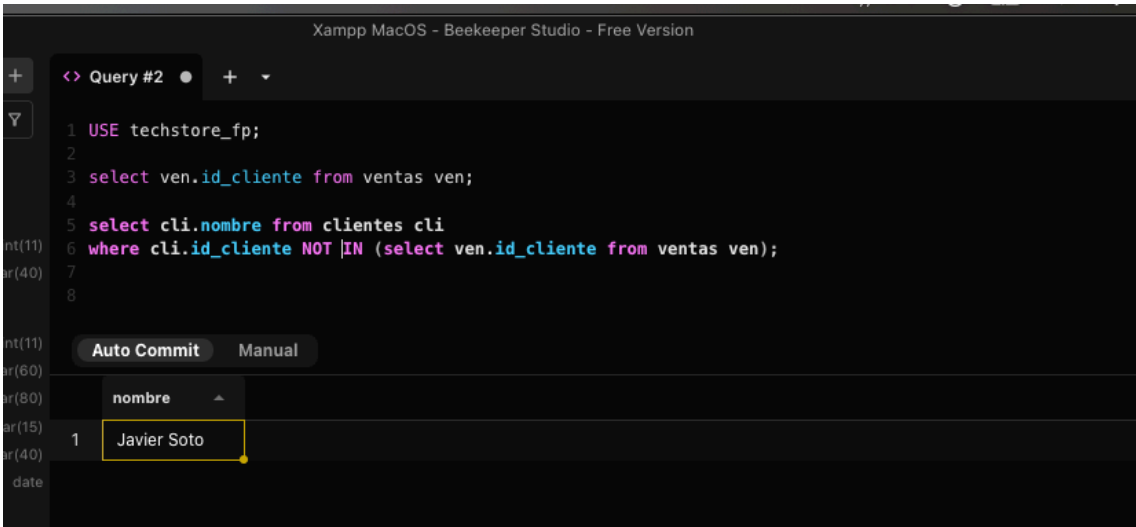


```
1 USE techstore_fp;
2
3 select ven.id_cliente from ventas ven;
4
5 select cli.nombre from clientes cli
6 where cli.id_cliente IN (select ven.id_cliente from ventas ven);
7
8
```

Auto Commit Manual

	nombre
1	Ana Ruiz
2	Luis Pérez
3	María Díaz
4	Carmen León

18. Clientes que NO han comprado nunca (NOT IN).



```
1 USE techstore_fp;
2
3 select ven.id_cliente from ventas ven;
4
5 select cli.nombre from clientes cli
6 where cli.id_cliente NOT IN (select ven.id_cliente from ventas ven);
7
8
```

Auto Commit Manual

	nombre
1	Javier Soto

19. Productos que se han vendido alguna vez.

The screenshot shows the Beekeeper Studio interface with a SQL query executed. The query selects the names of products that have been sold at least once by joining the 'productos' table with the 'detalle_venta' table. The results are displayed in a table with 8 rows.

```
1 USE techstore_fp;
2
3 select dv.id_producto from detalle_venta dv;
4
5 select prod.nombre from productos prod
6 where prod.id_producto IN (select dv.id_producto from detalle_venta dv);
7
8
```

	nombre
1	iPhone 13
2	Samsung A54
3	Xiaomi Redmi Note 12
4	Portátil Lenovo i5
5	MacBook Air M1
6	Auriculares Bluetooth
7	Cargador USB-C 65W
8	SSD 1TB

20. Productos que NO se han vendido nunca.

The screenshot shows the Beekeeper Studio interface with a SQL query executed. The query selects the names of products that have never been sold by selecting from the 'productos' table and excluding those found in the 'detalle_venta' table. The results are displayed in a table with 1 row.

```
1 USE techstore_fp;
2
3 select dv.id_producto from detalle_venta dv;
4
5 select prod.nombre from productos prod
6 where prod.id_producto NOT IN (select dv.id_producto from detalle_venta dv);
7
8
```

	nombre
1	RAM 16GB DDR4

21. Productos con precio superior al precio medio.

The screenshot shows the Beekeeper Studio interface with a SQL query executed. The query calculates the average price of all products and then selects the names of products whose price is greater than this average. The results are displayed in a table with 3 rows.

```
1 USE techstore_fp;
2
3 select AVG(pro.precio) from productos pro;
4
5 select pro.nombre, pro.precio from productos pro
6 where pro.precio > (select AVG(pro.precio) from productos pro);
7
8
```

	nombre	precio
1	iPhone 13	599.00
2	Portátil Lenovo i5	749.00
3	MacBook Air M1	899.00

22. Ventas cuyo total sea superior a la media de ventas.

Xampp MacOS - Beekeeper Studio - Free Version

Query #2

```
1 USE techstore_fp;
2
3 select AVG(dv.cantidad * dv.precio_unitario) from detalle_venta dv;
4
5 select * from ventas ven
6 join detalle_venta dv on dv.id_venta=ven.id_venta
7 where dv.cantidad*dv.precio_unitario > (select AVG(dv.cantidad * dv.precio_unitario) from detalle_venta dv)
8
```

Auto Commit Manual Save

	id_venta	fecha	id_cliente	metodo_pago	id_venta	id_producto	cantidad	precio_unitario
1	1	2025-04-10	1	TARJETA	1	1	1	599.00
2	3	2025-04-12	1	EFFECTIVO	3	2	1	329.00
3	4	2025-04-15	3	TARJETA	4	4	1	749.00
4	5	2025-04-18	5	TARJETA	5	5	1	899.00

23. Cliente que más ha gastado (una sola fila).

24. Productos vendidos en la misma venta que el "iPhone 13".

Soluciones (Nivel 3)

-- 17

```
SELECT *  
FROM clientes  
WHERE id_cliente IN (SELECT id_cliente FROM ventas);
```

-- 18

```
SELECT *  
FROM clientes  
WHERE id_cliente NOT IN (SELECT id_cliente FROM ventas);
```

-- 19

```
SELECT *  
FROM productos  
WHERE id_producto IN (SELECT id_producto FROM detalle_venta);
```

-- 20

```
SELECT *  
FROM productos  
WHERE id_producto NOT IN (SELECT id_producto FROM detalle_venta);
```

-- 21

```
SELECT *  
FROM productos  
WHERE precio > (SELECT AVG(precio) FROM productos);
```

-- 22

```
SELECT t.id_venta, t.total_venta
```

```

FROM (
    SELECT id_venta, SUM(cantidad * precio_unitario) AS total_venta
    FROM detalle_venta
    GROUP BY id_venta
) t
WHERE t.total_venta > (
    SELECT AVG(x.total_venta)
    FROM (
        SELECT id_venta, SUM(cantidad * precio_unitario) AS total_venta
        FROM detalle_venta
        GROUP BY id_venta
    ) x
);

```

-- 23 (cliente que más ha gastado)

```

SELECT c.id_cliente, c.nombre, tot.gasto_total
FROM clientes c
JOIN (
    SELECT v.id_cliente, SUM(dv.cantidad * dv.precio_unitario) AS gasto_total
    FROM ventas v
    JOIN detalle_venta dv ON v.id_venta = dv.id_venta
    GROUP BY v.id_cliente
) tot ON c.id_cliente = tot.id_cliente
ORDER BY tot.gasto_total DESC
LIMIT 1;

```

-- 24 (productos vendidos en la misma venta que iPhone 13)

```

SELECT DISTINCT p2.nombre
FROM detalle_venta dv2

```



```
JOIN productos p2 ON dv2.id_producto = p2.id_producto
WHERE dv2.id_venta IN (
  SELECT dv.id_venta
  FROM detalle_venta dv
  JOIN productos p ON dv.id_producto = p.id_producto
  WHERE p.nombre = 'iPhone 13'
);
```